Research Training: Smart Grids AI

Session 1: Introduction, Research Taste, & The Matrix

Instructor / Mentor

Kiki Research Training Program

Week 0

Contents

1. Course Intro & Philosophy
2. The Feynman Philosophy
3. Syllabus Walkthrough
4. Interactive: The Graph Theory Intuition
5. Vibe Coding Demo

Course Intro & Philosophy

Welcome to Smart Grids AI

Core Objectives

- **Programming Fundamentals:** Learn by doing.
- **Mathematical Understanding:** Focus on intuition, not memorization.
- **AI Collaboration:** Master "Vibe Coding" with LLMs.
- **Research Mindset:** Cultivate the "taste" of a true researcher.

The ultimate goal is **understanding**, not memorizing exact formulas. We are here to train future leaders, not human calculators.

Cultivating "Research Taste"

A researcher's most valuable asset is their *taste*. It dictates what they spend their life studying.

- **What questions CAN or CANNOT be answered?** (Feasibility)

Cultivating "Research Taste"

A researcher's most valuable asset is their *taste*. It dictates what they spend their life studying.

- **What questions CAN or CANNOT be answered?** (Feasibility)
- **What questions are WORTH answering?** (Impact & Novelty)

Cultivating "Research Taste"

A researcher's most valuable asset is their *taste*. It dictates what they spend their life studying.

- **What questions CAN or CANNOT be answered?** (Feasibility)
- **What questions are WORTH answering?** (Impact & Novelty)
- **HOW can they be answered?** (Methods & Tools)

Cultivating "Research Taste"

A researcher's most valuable asset is their *taste*. It dictates what they spend their life studying.

- **What questions CAN or CANNOT be answered?** (Feasibility)
- **What questions are WORTH answering?** (Impact & Novelty)
- **HOW can they be answered?** (Methods & Tools)
- **WHO can answer them?** (Leadership & Delegation)

Cultivating "Research Taste"

A researcher's most valuable asset is their *taste*. It dictates what they spend their life studying.

- **What questions CAN or CANNOT be answered?** (Feasibility)
- **What questions are WORTH answering?** (Impact & Novelty)
- **HOW can they be answered?** (Methods & Tools)
- **WHO can answer them?** (Leadership & Delegation)

For Future Leaders

You don't need to do every calculation yourself, but you must know *how* to direct the people (or AI) who do.

Weekly Course Structure

120 minutes per week, divided into three focused sessions:

1. **Session 1 (40 min) – Scenarios & Challenges:** Interactive demos. Experience the problem before learning the solution.
2. **Session 2 (40 min) – Mathematical Principles:** Why the tools work. Focus on deep intuition.
3. **Session 3 (40 min) – Vibe Coding:** You lead the logic. AI assists with the syntax. Solve advanced challenges together.

Homework A: Manual Programming Training

The Rules

- Mentors provide core logic code with `TODO` placeholders.
- **Strictly NO AI allowed.**
- You must complete the code by reading official documentation.

Why?
Before you can direct AI (Vibe Coding), you must build foundational "coding muscles."
You need to know what good code looks like.

Homework B: The Feynman Technique

The Assignment

Post a learning note about the week's content on social media (IG, X, WeChat Official Account, Xiaohongshu, etc.).

Why?

- Teaching a concept to others leads to the deepest understanding.
- Improves your public visibility and scientific communication.
- Crucial for the American academic and professional environment.

The Feynman Philosophy

Who was Richard Feynman?

- **Physics Giant:** 1965 Nobel Laureate, pioneer of Quantum Electrodynamics (QED).
- **Manhattan Project:** Key researcher at age 24.
- **"The Great Explainer":** Made complex physics accessible (e.g., *The Feynman Lectures*).
- **Challenger Disaster:** Demonstrated the O-ring failure on live TV with just ice water.

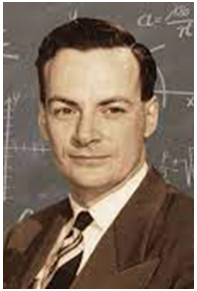


Figure: Richard Feynman

Feynman's Anecdotes: The "Scientific Trickster"

Feynman's life proved that **curiosity** is the greatest driver of learning.

- **Professional Safecracker:** Bored by Manhattan Project security, he taught himself to crack safes, leaving prank notes in top-secret military vaults.
- **Unconventional Workspaces:** He often worked in noisy cabarets/bars. The chaotic environment helped him focus. He even explained physics to the staff!
- **Samba Drummer:** While in Brazil, he didn't just teach physics; he joined a local band and marched in the Carnival as a professional percussionist.

Feynman's Anecdotes: Defying Expectations

- **The Reluctant Laureate:** When he won the Nobel Prize, he almost rejected it to avoid the media attention. He only accepted when warned that rejecting it would cause an even bigger media circus!
- **Cross-Disciplinary Hacker:** As a grad student, he crashed graduate biology classes. He shocked the biology students by asking for a "map of a cat" (an anatomical chart). He never let boundaries stop his curiosity.

"I would rather have questions that can't be answered than answers that can't be questioned."

The Feynman Technique

The Core Philosophy

"If you cannot explain it simply, you don't understand it."

The Feynman Technique is a mental model for learning anything efficiently. It forces you to strip away jargon and confront your own ignorance honestly.

The 4 Steps of the Feynman Technique

1. **Choose a Concept:** Write the topic on a blank sheet of paper.
2. **Teach it to a Child:** Explain it using the simplest language possible. *Jargon hides a lack of understanding.*
3. **Review and Refine:** When you get stuck or logic breaks down, go back to the source material. Fill the gap.
4. **Simplify and Analogy:** Create an analogy. (e.g., Voltage is like "water pressure"; Current is the "water flow").

Syllabus Walkthrough

Syllabus Walkthrough: Why learn this?

Goal: Prepare you for high-level Smart Grid System Research.

Modern electrical grids are the largest, most complex machines ever built by humans. To study them, we need a bridge between:

- **Math:** To model the physical world.
- **Code:** To process millions of data points.
- **AI:** To predict the unpredictable (weather, human behavior, failures).

Phase 1 & 2: Foundations & Signals

Phase 1: Representation (Wk 1-4)

- Everything is a Vector.
- Graph Theory (Grid Skeletons).
- Database logic (SQL & Lit Search).
- Linear Algebra (The Language of Change).

Phase 2: Dynamics & Signals (Wk 5-8)

- Calculus Intuition (Derivatives/Integrals).
- Literature Taste (Reading abstracts).
- Signal Processing (FFT, Frequencies).
- Graph Signal Processing (GSP).

Phase 3, 4 & 5: Optimization, AI, and Output

Phase 3: Optimization & Games (Wk 9-12)
Convex optimization (budgeting), Game Theory (cooperation vs. competition), and handling the stochasticity of renewable energy.

Phase 4: Core AI Transformation (Wk 13-17)
From linear regression to SVMs, Neural Networks, PCA, and LSTMs for time-series memory (The "Duck Curve").

Phase 5: Research Output (Wk 18-20)
Peer review critique, building a grid "Digital Twin", and a final presentation to an Expert Committee.

Interactive: The Graph Theory Intuition

Interactive Game: The Memory Box Budget

Imagine you have physical "boxes".
Each box can store exactly *one* number or English word.
Your memory budget is strictly limited!

Task 1: Summing Two Numbers

Task: Calculate $A + B = C$.
How many boxes do you need to execute and store this?

Task 1: Summing Two Numbers

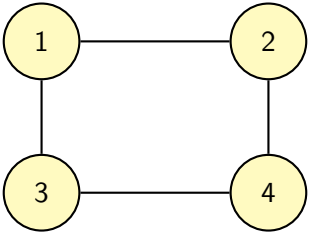
Task: Calculate $A + B = C$.
How many boxes do you need to execute and store this?

- 1 box for A
- 1 box for B
- 1 box for the result C

Total: 3 Boxes. Easy!

Task 2: A Small Grid System

Task: We have a town with 4 power nodes. How do we store the information of *who is connected to whom* using the least number of boxes?



The Naive Approach vs. Graph Theory

Naive Approach (Text List):

- "Node 1 is connected to Node 2" (Takes many boxes for strings/text)
- "Node 1 is connected to Node 3"

Terrible for computers. Costs too much memory!

The Naive Approach vs. Graph Theory

Naive Approach (Text List):

- "Node 1 is connected to Node 2" (Takes many boxes for strings/text)
- "Node 1 is connected to Node 3"

Terrible for computers. Costs too much memory!

Graph Theory Approach: Instead of text, we use a grid of boxes containing just 1 (Connected) or 0 (Not Connected).

The Adjacency Matrix

We represent the 4 nodes using a 4×4 Matrix.

- Row represents "From"
- Column represents "To"
- 1 means a line exists.
- 0 means no line.

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

How Computers Speak "Matrix"

How do different programming languages store these boxes?

- **Python (NumPy):** A list of lists. `adj = [[0,1,1,0], [1,0,0,1], ...]`
- **MATLAB:** Native matrix object. `adj =[0 1 1 0; 1 0 0 1; ...]`
- **R:** DataFrames or Matrix objects.

Why do we care?

Computers are highly optimized to process matrices in parallel. Doing math on a matrix is thousands of times faster than using `for`-loops!

Math in Action: Engineering Applications

Why is the Adjacency Matrix crucial for Smart Grids?

1. Power Flow Calculation: If we multiply the Adjacency Matrix by a vector of Voltages, we can instantly calculate power flows across the entire town.

2. Cascading Failures: If Node 2 fails, we delete row 2 and column 2. By doing matrix multiplication, we can simulate if the rest of the grid will collapse.

Step 1: The Laplacian Matrix (L)

The Adjacency Matrix A only shows connections. To do physics, we need the **Laplacian Matrix**:

$$L = D - A$$

- D **(Degree Matrix):** A diagonal matrix where D_{ii} is the number of lines connected to node i.
- A **(Adjacency Matrix):** Our $0/1$ map of connections.

$$\boxed{D} - \boxed{A} = \boxed{L}$$

The "Stiffness" of the Grid

Step 2: The Power Flow Equation

In a DC approximation (Linear Flow), the relationship between injected current (I) and voltage (V) at every node is:

$$\vec{I} = L \cdot \vec{V}$$

For a single node i, this expands to Kirchhoff's Current Law:

$$I_i = \sum_{j \in \text{neighbors}} (V_i - V_j) \cdot y_{ij}$$

- L acts as the "operator" that transforms voltages into flows.
- **Matrix Vector Multiplication:** This is why "Everything is a Vector."

Step 3: Simulating Cascading Failures

What happens when Node k fails? We modify the matrix:

$$L_{\text{new}} = L \setminus \{\text{row } k, \text{col } k\}$$

Mathematical Impact:

- The dimension drops from N to $N-1$.
- We check the **Algebraic Connectivity** (the second smallest eigenvalue λ_2).
- If $\lambda_2 = 0$, the grid has split into "islands."

$$\begin{bmatrix} L_{11} & L_{12} & L_{13} \\ L_{21} & L_{22} & L_{23} \\ L_{31} & L_{32} & L_{33} \end{bmatrix}$$

FAIL

Figure: Topology change via matrix reduction

Why this matters for Research

The Researcher's "Taste"

By looking at the eigenvalues of L, you can tell if a grid is "strong" or "fragile" without even looking at a map.

- **Optimization:** We design grids to maximize λ_2 (robustness).
- **AI Training:** We feed these matrices into Neural Networks to predict which node failure causes the most damage.
- **Speed:** Calculating $L \cdot V$ is much faster than solving thousands of non-linear equations.

Everything is a Vector

Wait, a 4×4 matrix uses 16 boxes. What if we just line them all up?

$$\begin{array}{|c|c|}\hline 0 & 1 \\ \hline 1 & 0 \\ \hline \end{array} \xrightarrow{\text{Flatten}} \begin{array}{|c|c|c|c|}\hline 0 & 1 & 1 & 0 \\ \hline \end{array}$$

Core Intuition for Week 0:
A matrix is just a folded vector. A vector is just a point in high-dimensional space.
Everything is a vector!

Vibe Coding Demo

What is Vibe Coding?

The Future of Programming

You don't memorize syntax. You direct the logic.
You tell the AI *what* to build and *how* it works conceptually. The AI writes the boilerplate.

Let's use Python to build the exact Adjacency Matrix we just saw, and let an AI module calculate the grid connections!

Scaling Up: From 4 Nodes to a Town

- **Manual Era:** Drawing 4 nodes and a 4×4 matrix by hand.
- **Research Era:** Managing 20+ nodes, substations, and hundreds of lines.
- **The Challenge:** How do we find the "weakest link" in a complex web?

The Vibe Coding Approach

We don't define every connection manually. We tell the AI the *topology* we want (e.g., "A small-world network representing a suburban grid") and ask it to simulate a failure.

Visualizing the Town Grid

System Properties:

- 20 Local Transformers (Nodes)
- Redundant Mesh Topology
- Real-time "Edge" weights (Power flow)

In research, we look for "Clusters" and "Hubs" that, if broken, cause a blackout.

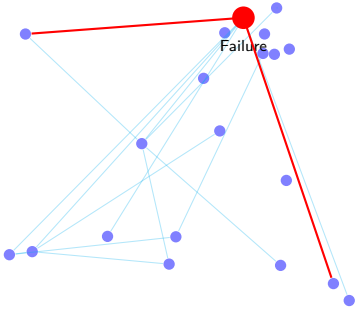


Figure: Simulated 20-node Town Grid

Vibe Code: Identifying Vulnerabilities

```python
import networkx as nx
import matplotlib.pyplot as plt

# "Vibe": AI, create a complex power grid with 20 nodes
G = nx.powerlaw_cluster_graph(n=20, m=2, p=0.1)

# "Vibe": Find the most critical substation (Centrality)
centrality = nx.betweenness_centrality(G)
critical_node = max(centrality, key=centrality.get)

# "Vibe": Simulate a failure and check connectivity
G.remove_node(critical_node)
is_stable = nx.is_connected(G)

print(f"Critical Node {critical_node} failed!")
print(f"Is the grid still stable? {is_stable}")
```

Homework Reminder

Week 0 Action Items

1. **Manual Coding (Homework A):** None
2. **Feynman Technique (Homework B):** Take what you learned today (Vectors, Matrices, or Feynman himself) and explain it on social media.

Let's get started!